

EDGE INTELLIGENCE

LAB - 2

PRADHEESH RAJ R

25MML0023

TASK-1:

Analyzing an image dataset-Mnist , applying basic preprocessing operations, using either ANN or CNN. Saving the model using pickle model.

Key takeaway: understanding the dataset and saving the model using pickle module.

```
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.utils import to_categorical
```

```
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

```
print(X_train.shape)
print(X_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ————— 0s 0us/step
(60000, 28, 28)
(10000, 28, 28)
```

```
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)
```

```
# Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0
```

```
# One-hot encode labels
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
```

```
model = Sequential()

model.add(Conv2D(32, kernel_size=(3,3), activation='relu', input_shape=(28,28,1)))
model.add(MaxPooling2D(pool_size=(2,2)))

model.add(Conv2D(64, kernel_size=(3,3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2,2)))

model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(10, activation='softmax'))

model.summary()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequential models, prefer using an 'Input(shape)'
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)
Non-trainable params: 0 (0.00 B)

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
model.fit( X_train, y_train, epochs=5,batch_size=64)
```

```
Epoch 1/5
938/938 ————— 46s 47ms/step - accuracy: 0.8916 - loss: 0.3831
Epoch 2/5
938/938 ————— 43s 46ms/step - accuracy: 0.9836 - loss: 0.0518
Epoch 3/5
938/938 ————— 82s 46ms/step - accuracy: 0.9888 - loss: 0.0334
Epoch 4/5
938/938 ————— 83s 47ms/step - accuracy: 0.9918 - loss: 0.0232
Epoch 5/5
938/938 ————— 82s 47ms/step - accuracy: 0.9945 - loss: 0.0174
<keras.src.callbacks.history.History at 0x7e87f7eb8530>
```

```
test_loss, test_accuracy = model.evaluate(X_test, y_test)
print("Test Accuracy:", test_accuracy)
```

```
313/313 ————— 3s 8ms/step - accuracy: 0.9871 - loss: 0.0365
Test Accuracy: 0.9904000163078308
```

```
import pickle

# Save model architecture (config)
model_config = model.to_json()

# Save model weights
model_weights = model.get_weights()

with open("cnn_model.pkl", "wb") as f:
    pickle.dump((model_config, model_weights), f)

print("Model saved using pickle")
```

Model saved using pickle

```
import pickle
from tensorflow.keras.models import model_from_json

with open("cnn_model.pkl", "rb") as f:
    model_config, model_weights = pickle.load(f)

# Rebuild model
loaded_model = model_from_json(model_config)
loaded_model.set_weights(model_weights)

# Compile again
loaded_model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

print("Model loaded successfully")
```

Model loaded successfully

Task - 2:

Create account in edge impulse, go to data acquisition, Choose connect data option, scan QR using mobile, Select Label data before clicking a photo, Split the clicked photos into train and test data.

The screenshot displays the Edge Impulse Studio web interface. The browser address bar shows the URL: `studio.edgeimpulse.com/studio/872577/acquisition/training?page=1`. The user is logged in as PRADHEESHRAJR / pradheeshrajr_25MMLOO23 (PERSONAL). The target device is set to Cortex-M4F 80MHz.

The interface is divided into a left sidebar and a main content area. The sidebar contains the following navigation items:

- Dashboard
- Devices
- Data acquisition (selected)
- Experiments
- EON Tuner
- Impulse design (dropdown menu)
 - Create impulse
 - Live classification
 - Model testing
 - Post-processing
- Upgrade Plan (Get access to higher job limits and more collaborators. View plans)

The main content area has a top navigation bar with the following tabs: Dataset (selected), Data sources, Synthetic data, Labeling queue (32), and AI labeling (NEW). Below the tabs, there are three main sections:

- DATA COLLECTED**: 32 items
- TRAIN / TEST SPLIT**: -
- Collect data**: Connect a device to start building your dataset.

The **Dataset** section shows a table with the following columns: SAMPLE NAME, LABELS, and ADDED. The table contains 8 rows of data:

SAMPLE NAME	LABELS	ADDED
Waterbottle.6ebbf6u2	-	Today, 17:15:...
Waterbottle.6ebbf26v	-	Today, 17:15:...
Waterbottle.6ebb8euc5	-	Today, 17:15:...
Pen.6ebb9cdt	-	Today, 17:12:...
Pen.6ebb99uh	-	Today, 17:12:...
Pen.6ebb8om1	-	Today, 17:12:...
Pen.6ebb8nsn	-	Today, 17:12:...

A dark blue button labeled "Click on a sample to load..." is visible on the right side of the dataset table.